ISA 401: Business Intelligence & Data Visualization

02: Git Foundations + Reproducibility

Fadel M. Megahed, PhD

Raymond E. Glos Professor in Business
Farmer School of Business
Miami University

@Fadel Megahed
fmegahed
fmegahed@miamioh.edu
Automated Scheduler for Office Hours

Fall 2026

Before We Start

04:00

1 Join today's Zoom

Use the Zoom link in the **Week 0 Module** on Canvas. **Mute** your mic and **turn off** your video.
Please log in from the **lab's computer** so you have the option to **share your screen** if you need help debugging an error.

2 Install GitHub Desktop

Run the **GitHub Desktop** installer: it is on the lab computer's **Desktop**, or type *GitHub Desktop* in the **Windows search bar**.
It installs in about a minute and opens by itself.

3 Sign in

Sign in to GitHub.com with the account you created in **Assignment 0**.
Your browser opens, you authorize, and it hands you back to Desktop.

4 Configure Git

When asked, keep the **name** and **email** you use on GitHub, then click **Finish**.
This is what stamps your name on every snapshot you save.

Done early? **Help the person next to you.** Stuck? **Share your screen on Zoom** and we fix it together. Nobody sinks in this class; **everyone swims.**

Quick Refresher from Last Class

THE ANALYTICS JOURNEY

Pre-analytics get and clean the data
Descriptive what happened?
Predictive what will happen?
Prescriptive what should we do?

BI lives in the first two, and feeds the rest.

ONE JOB AD, THREE SHAPES

Unstructured the ad text
Semi-structured the skills JSON
Structured the database row

Most of this course moves data from top to bottom.

HABITS WE PRACTICED

Unit of analysis: rows ≠ crashes
Counts vs. rates: a busy weekday is not a risky one
Provenance: who collected this, and how?
Encodings: position, length, color, shape

Today we start doing all of this in code.

Learning Objectives for Today's Class

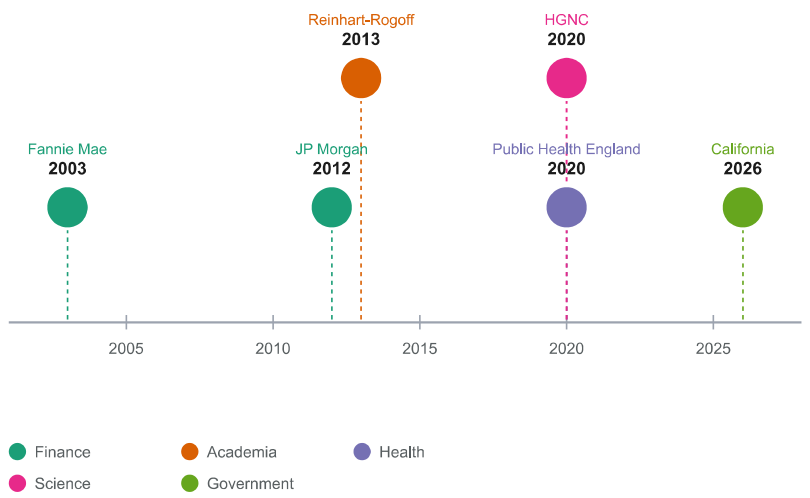
- **Explain** why scripted languages enable **reproducible analysis**.
- **Initialize and use Git** for version control.

PART 1	PART 2
Why scripts?	**Git + GitHub**
Spreadsheets that broke, and code that re-runs	Snapshots, commits, and your first repo

Setup-heavy on purpose: every script you write this semester lives in the repository you clone in Part 2.

Why Scripts?

Spreadsheets That Made the News



Click a year on the left.

Each write-up ends with a link to the primary source (a company report, a journal article, or the original news coverage), so you can verify every claim.

- **There is always more:** a 2024 review of 35 years of studies reports that **94% of the business spreadsheets audited in those studies contained errors** (Poon et al. (2024), *Frontiers of Computer Science*).
- The Guardian keeps a running bloopers reel of spreadsheet errors going back to 1985.

The Same Chart in a Spreadsheet and in R

The crashes-by-weekday chart from Class 01, built twice from the same CSV. The chart is identical. The **record of how it was made** is not.

IN A SPREADSHEET

1. Download the crash CSV from the open-data portal
2. Open it in Excel
3. One crash spans several rows: Data → Remove Duplicates on `instanceid`
4. Add a weekday column with `=TEXT(B2,"ddd")`, drag it down
5. Insert a pivot table; count rows by weekday
6. Insert a bar chart; paste it into PowerPoint

Now do it by month instead of weekday. Which of these steps do you redo?

IN R

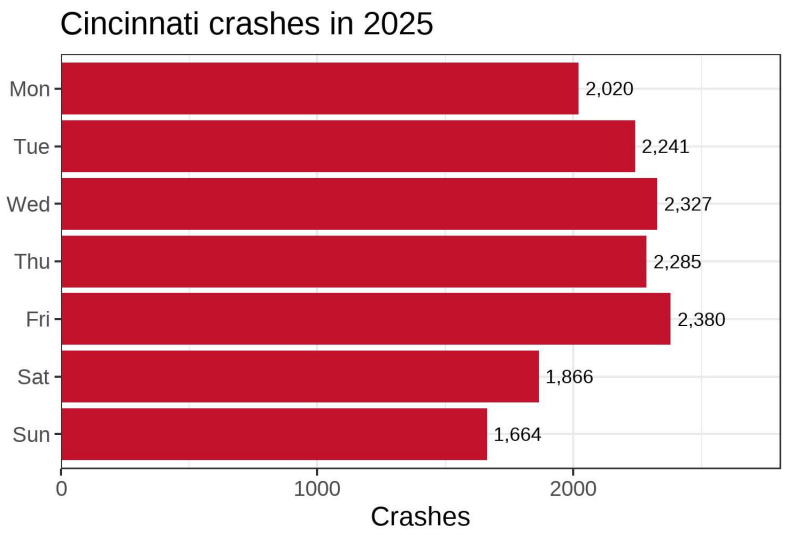
```
crashes = readr::read_csv("data/cincy_2025_crashes.csv") |>
  dplyr::mutate(
    datetime = lubridate::ymd_hms(crashdate),
    weekday  = lubridate::wday(datetime, label = TRUE)
  ) |>
  dplyr::distinct(instanceid, .keep_all = TRUE)

crashes |>
  dplyr::count(group = weekday) |>
  ggplot2::ggplot(
    ggplot2::aes(x = n, y = forcats::fct_rev(group))
  ) +
  ggplot2::geom_col(fill = "#C3142D") +
  ggplot2::labs(x = "Crashes", y = NULL,
                title = "Cincinnati crashes in 2025")
```

Changes to Unit of Analysis are Easier in R

By weekday | By month

```
crashes |>
  dplyr::count(group = weekday) |>
  ggplot2::ggplot(
    ggplot2::aes(x = n, y = forcats::fct_rev(group))
  ) +
  ggplot2::geom_col(fill = "#C3142D") +
  ggplot2::geom_text(
    ggplot2::aes(label = scales::comma(n)),
    hjust = -0.15, size = 3
  ) +
  ggplot2::scale_x_continuous(
    expand = ggplot2::expansion(mult = c(0, 0.18))
  ) +
  ggplot2::labs(x = "Crashes", y = NULL,
                title = "Cincinnati crashes in 2025")
```



Code Is Just Text

Programming languages are **languages**. Code is **just text**, which hands you two extremely powerful techniques:



Readable
You can read what someone did, top to bottom. Try that with a pivot table.

Open
An error message is text too: search it, or paste it into ChatISA.

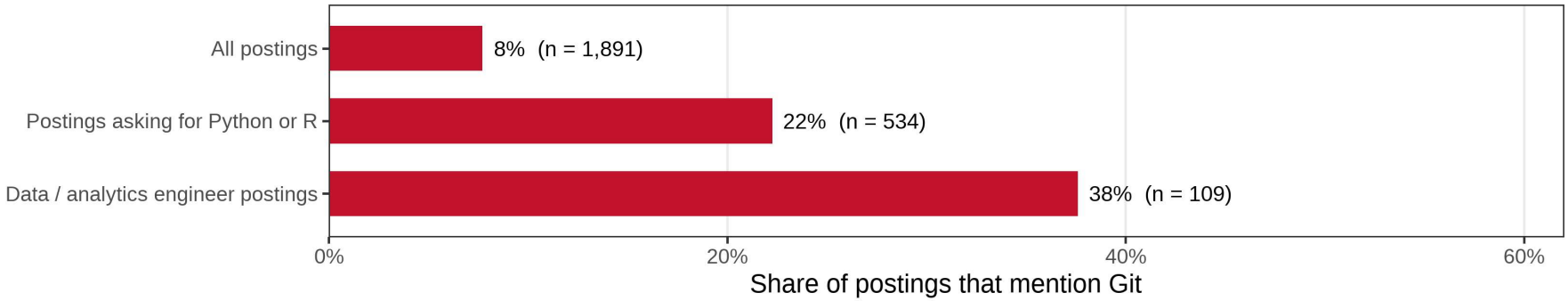
Reusable
New month, new file: change one line and re-run. Someone else gets the same result.

Diffable
Text can be compared line by line, so a tool can show you exactly what changed, and when. That tool is next.

Source: Based on Hadley Wickham's You Can't Do Data Science in a GUI, slides 14 to 17.

Git + GitHub

Git in Currently Open ISA Jobs



Employer	Role	Location	Posted
Apple	Product Data Scientist, Employee Experience & Pro...	Sunnyvale CA	Aug 19
Booz Allen Hamilton	Data Engineer	San Diego CA	Aug 05
CapTech Consulting	Data Analyst	Columbus OH	Aug 18
U.S. Soccer Federation	Data Scientist, Business Ventures	Atlanta GA	Aug 22

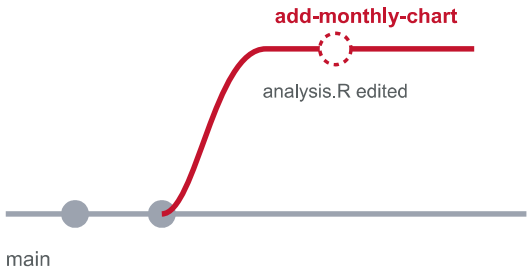
- **Chart:** every posting collected by ChatISA Job Scout from Jun 30 to Aug 23, 2026 (1,891 postings). A posting "mentions Git" if its skills tag includes version control or its text names git, GitHub, GitLab, or Bitbucket.
- **Table:** four of the 32 open postings that ask for **both R and Git** (links checked Aug 25, 2026). Chart and table are computed from the database by code inside this deck.

Git is Heavily Used by AI Harnesses

What an AI coding agent does when you hand it a task, as the vendors themselves describe it (sources below). The terminal is a simplified mock-up; the six steps loop. Click the animation to pause it.

1 · YOU	2 · AGENT	3 · AGENT	4 · AGENT	5 · AGENT	6 · YOU
Describe the task	Creates a branch	Edits the code	Commits a snapshot	Pushes and opens a PR	Review, then merge

```
you › Add a monthly crash chart to the report.
agent › git switch -c add-monthly-chart
Switched to a new branch 'add-monthly-chart'
agent › edit analysis.R
+  dplyr::count(group = month) |>
+  ggplot2::ggplot(ggplot2::aes(x = n, y = group)) +
+  ggplot2::geom_col()
3 lines added, 0 removed
```



- **Claude Code:** a mention of `@claude` on GitHub can "implement changes, and push commits" and "turn issues into pull requests" (Anthropic docs).
- **Codex:** works in an isolated copy of your repository, then lets you "open a pull request when the work is ready" (OpenAI docs).
- **GitHub Copilot coding agent:** makes "code changes on a branch" and opens a pull request for a person to review (GitHub docs).

Git vs GitHub

Git

A **program on your computer** that keeps **snapshots of a folder**.

Created in **2005** by Linus Torvalds to manage the Linux kernel's source code. Free and open source.

Works with no internet connection. GitHub Desktop, RStudio, and VS Code are all just buttons on top of it.

GitHub

A **website** that **hosts Git folders** so people can share, review, and publish them.

Owned by Microsoft since **2018**. More than **180 million users**. Free for what we do.

Your `business_intelligence` repo from Assignment 0 lives here. Employers look here.

Git is the tool. It runs on your machine and works offline.

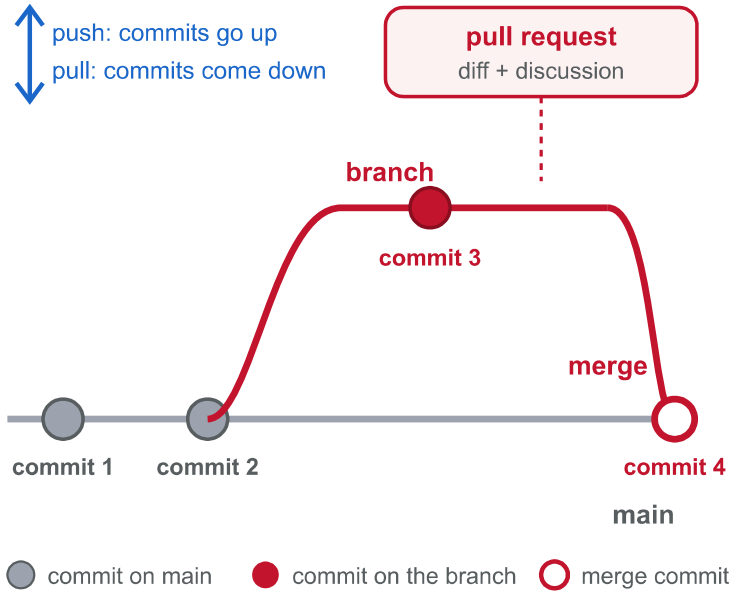
GitHub is the place. Copies live there so they can be shared, reviewed, and published.

You can use Git without GitHub. You cannot use GitHub without Git.

- **Git's origin:** the official Git book, A Short History of Git.
- **Ownership:** Microsoft completes GitHub acquisition, Oct 26, 2018.
- **Users:** GitHub Octoverse 2025, "180 million-plus developers."

Git's Terminology

Word	Means
repository (repo)	a folder Git watches
commit	a named snapshot of that folder
push / **pull**	upload / download snapshots to or from GitHub
branch	a parallel copy for work in progress
pull request	"please merge my branch," plus discussion
merge	accept it into `main`



What You Did in Assignment 0

On github.com, in your browser, you did three things. Each one has a Git word:

1 New repository

You named it `business_intelligence` and made it public.

That created a **repository**: a folder GitHub watches.

→

2 Add README.md

You wrote your name and the course topics into the file.

Git does not know about a file until it is committed.

→

3 Commit changes

The green button at the bottom of the editor.

That was your first **commit**: a named snapshot. GitHub made it for you, on its servers.

So far the repo exists in **one place: GitHub**. No computer has a copy, and every edit went through a web page. Real work (R scripts, data, plots) happens on a computer, so the next step is to bring the repo **down** to this one: **clone**. After that, three new verbs: **change, commit, push**.

Live Demo: Clone, Change, Commit and Push

Follow along on your lab computer. I do each step on the screen first, then you do it.

Steps | Checkpoint | Discussion

1 Clone. GitHub Desktop → **File → Clone repository** → pick `YOUR-USERNAME/business_intelligence`. Set **Local path** to `M:\isa401\` (your M: drive follows you to any lab computer; we open this folder in RStudio next). Click **Clone**.

2 Change. Repository → Show in Explorer. Open `README.md` in Notepad. Under *About Me* add one line, then save:

```
- **Tools this semester:** R, RStudio, Git
```

3 Commit. Back in Desktop, **look at the diff**: your new line is green, and the checkbox next to `README.md` is ticked. Summary: `Add tools line`. Click **Commit to main**.

4 Push. Click **Push origin**.

5 Verify. Repository → View on GitHub. Refresh. Your line is there and the repo now shows **2 commits**.

IF YOU FALL BEHIND

Say so, or **share your screen on Zoom**. Nobody moves on until everyone has pushed.

The usual issues:

- Desktop is signed in to the wrong account: **File → Options → Accounts**
- Notepad saved the file as `README.md.txt`: turn on file extensions in Explorer
- Repo not listed under Clone: paste the URL `https://github.com/YOUR-USERNAME/business_intelligence`

Publish a Career Portfolio with ChatISA

Steps | Poll | Discussion

1 Sign in

Go to **chatisa.fsb.miamioh.edu/portfolio** and **Sign in with GitHub**, authorizing ChatISA to act on your account.

→

2 Fill in the builder

Your **name**, a **headline**, a **photo**, and **2 to 3 projects**. Click **Generate**.

ChatISA drafts the page text with AI, creates a repository named `portfolio` on your GitHub, and publishes it.

→

3 Open it

On your phone: `https://YOUR-USERNAME.github.io/portfolio/` (allow about a minute).

A website with your name on it, hosted by GitHub, from a repo you now own.

What you have now is a first draft. In **Assignment 02** you make it yours, one commit at a time: read every AI-generated sentence and edit it so it is accurate and sounds like you, add your contact links, keep only the projects you would defend in an interview, and reorder the sections (it is your name on the page, and recruiters will read it).

Then answer the poll on the next tab: **which Git verbs did ChatISA just run on your behalf?** Full walkthrough of the builder: youtu.be/I85EeVFfkEo.

Text Files vs Binary Files

TEXT

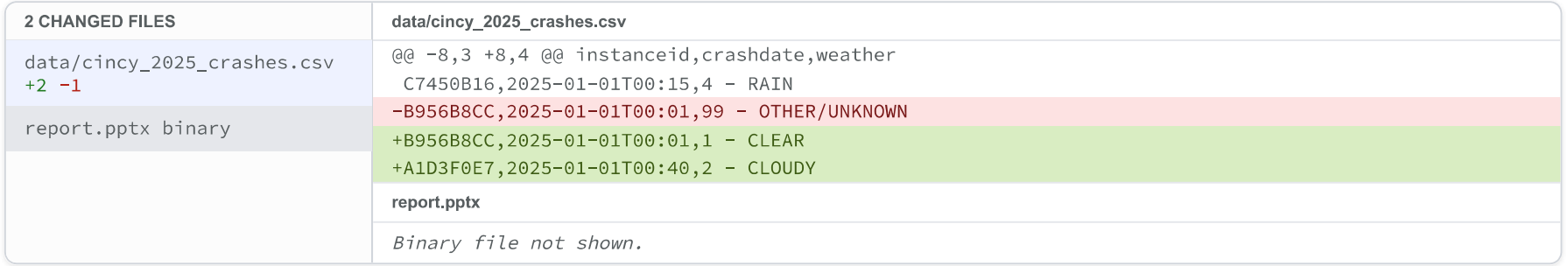
Code, CSV, Markdown, .R, .Rmd, .qmd, HTML

Git stores the **lines**. Desktop shows a red/green diff. The history is readable years later. Two people can merge, line by line.

BINARY

.xlsx, .pptx, .docx, .png, .pdf

Git stores the **whole file**, every commit. Desktop says "Binary file not shown." No line-by-line merge: someone loses. The repo gets heavy, fast.



Rule of thumb: data as CSV, code and notebooks as text, and keep the big binaries out of the repo (share them from Google Drive and link to them).

Fixing Mistakes

Before you push: Undo

A commit you have not pushed is still **private to this computer**. GitHub has never seen it.

In Desktop, under the commit button: click **Undo**. The commit disappears and the changes come back to the left pane, so you can fix them and commit again.

Typo in the message? Committed the wrong file? Undo.

After you push: Revert

Once pushed, the commit is on GitHub, and possibly on a teammate's machine. **Git never deletes history.**

In Desktop's **History** tab: right-click the commit → **Revert changes in commit** → Push origin. Revert is a **new commit that flips the old one**. Both stay in the log.

Deleted a section by accident and already pushed? Revert.

Git is a time machine. Nothing you commit is ever really lost. That is exactly why you should **commit early and commit often**, and stop keeping `final_v3_REALLY_final.xlsx` on your desktop. The filename was doing this job, badly.

Recap

Summary of Main Points

By now, you should be able to do the following:

- **Explain** why a script beats a spreadsheet for **reproducible analysis**: the steps are written down, run in order, and run again on new data after changing one line.
- **Use** the Git verbs **clone, commit, push**, and say what a **repository** is and what **GitHub** adds to Git.

Skills you practiced along the way: reading a **diff**, telling **text files** from binary files, and undoing a commit before and after it is pushed.

Before You Go

Which concept from today still needs clarification? Vote, and we go through the top answers before you leave.



Review and Clarification

- **Class Notes**: Take some time to revisit your class notes for key insights/concepts.
- **Zoom Recording**: The recording of today's class will be made available on Canvas approximately 3-4 hours after the session ends.
- **AI Recaps:** make your own with Gemini's NotebookLM, as I did for Class 01 (see the right).
- **Questions**: Do not let confusion pile up.

Make your own recap in NotebookLM

- Go to notebooklm.google.com and sign in with your Miami Google account.
- Add today's slide deck as a source: paste the deck's address from Canvas as a **Website** source.
- Click **Audio Overview** for a podcast, or **Video Overview** for a narrated video. Both take a few minutes to generate.

What it looks like: the Class 01 video and podcast. Treat them as a study aid, not a substitute for the slides: the AI summarizes what is on the page and can misstate details.

🎯 Assignments 🎯

- **Assignment 02: Make the Portfolio Yours.** Clone the `portfolio` repository ChatISA created for you, make it an RStudio project, push one mechanical commit, then edit every sentence you would not say to a recruiter, add your contact links, and reorder the sections. Submit the address of the live site.
- **Assignment 03: Git, RStudio, and Your Portfolio.** A short Canvas quiz on what you just did. Take it after your portfolio edits are pushed; GitHub Desktop's History tab answers half of it.

> **Do not regenerate the site on ChatISA.** Generating again overwrites the repository and erases your edits. From here on, the repository is yours: edit, commit, push.